LECTURE

BERT in depth

Pre-train once on raw text. Fine-tune for any task.

Based on Devlin et al. (2018), “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”

The idea: pre-train, then fine-tune

1. Pre-train one deep **bidirectional** encoder on unlabeled text.
2. Add **one small output layer** for each task.
3. Fine-tune **all** the weights on the task's labeled data.

Result in 2018: new state of the art on **11 NLP tasks** with one model.

Two steps, one model

	Pre-training	Fine-tuning
Data	Unlabeled text	Labeled task data
Tasks	Masked LM + next sentence	GLUE, SQuAD, SWAG, NER
New weights	—	One small output layer
Cost	4 days, 16–64 TPU chips	≤ 1 hour, one Cloud TPU
How often	Once	Once per task

Architecture: a stack of encoders

	BASE	LARGE
Layers L	12	24
Hidden H	768	1024
Heads A	12	16
Params	110M	340M

Input vectors

↓

Self-attention: all tokens

↓

Add + LayerNorm

↓

Feed-forward (4H), GELU

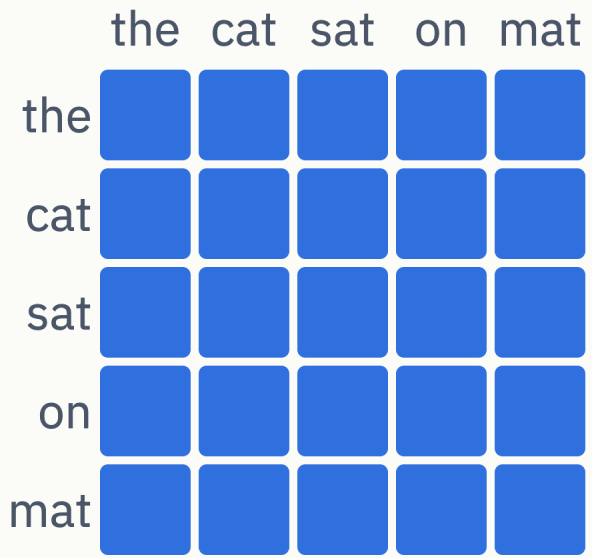
↓

Add + LayerNorm **× L**

BERT-base has the same size as OpenAI GPT, to compare fairly.

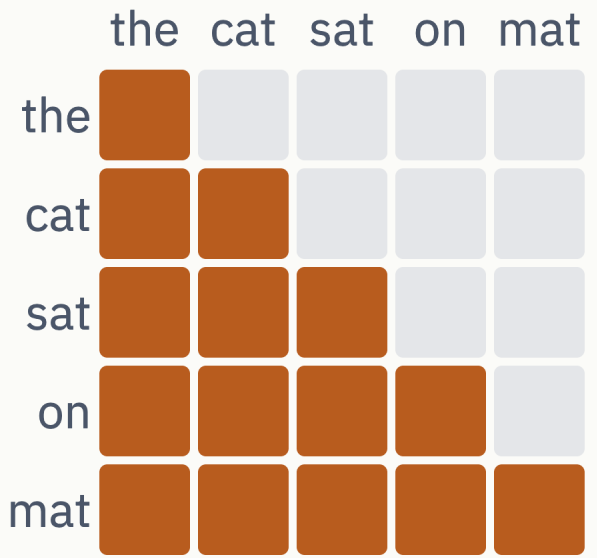
The key idea: read both ways

BERT



Reads **every** token.

OpenAI GPT



Reads only **earlier** tokens.

Row = the token that reads. Grey = blocked.

Input: three vectors per token

Token	[CLS]	my	dog	is	cut e	[SEP]	he	likes	play	##ing	[SEP]
Segment	A	A	A	A	A	A	B	B	B	B	B
Position	0	1	2	3	4	5	6	7	8	9	10

```
x_i = token_emb[tok_i] + segment_emb[A or B] + position_emb[i]
```

[CLS] = summary

[SEP] = separator

##ing = sub-word piece

Pre-training data: raw text only

Corpus	Size	What is kept
BooksCorpus	800M words	Full books
English Wikipedia	2,500M words	Text passages only

Wikipedia: **lists, tables and headers are removed.**

Keep **whole documents**, not shuffled sentences. Next-sentence prediction needs long runs of real, consecutive text.

No human labels. The text itself gives every target.

Building one training example

1. Take text span A from a document.
2. Span B: **50%** the real next text, **50%** random text.
3. Join as [CLS] A [SEP] B [SEP], at most 512 tokens.
4. Select **15%** of the tokens to predict.

```
[CLS] the man went to [MASK] store [SEP] he bought a gallon [MASK]
milk [SEP]
```

Labels: **IsNext** · first [MASK] = **the** · second [MASK] = **of**

Task 1: masked language model

my dog is hairy → BERT selects **hairy** to predict.

How often	Input to BERT	Target
80%	my dog is [MASK]	hairy
10%	my dog is apple	hairy
10%	my dog is hairy	hairy

Why not always [MASK]? **[MASK] never appears in fine-tuning.**

Task 2: next sentence prediction

```
[CLS] the man went to [MASK] store [SEP] he bought a gallon [MASK]
milk [SEP]
```

IsNext

```
[CLS] the man [MASK] to the store [SEP] penguin [MASK] are flight ##less
birds [SEP]
```

NotNext

C (the [CLS] vector) → 2-class softmax. **Why:** QA and NLI need the link between two sentences.

Pre-training: the recipe

Setting	Value
Loss	MLM loss + NSP loss
Batch	256 sequences (128,000 tokens)
Steps	1,000,000 (about 40 epochs)
Length	128 tokens for 90% of steps, then 512
Optimizer	Adam, learning rate 1e-4, 10k warmup
Hardware	16 (BASE) or 64 (LARGE) TPU chips, 4 days

Fine-tuning: one recipe

1. Load the pre-trained BERT weights.
2. Format the input: one sentence or a pair.
3. Add one output layer. These are the **only new weights**.
4. Train **all** weights on the task's labeled data.

Batch 16 or 32 · learning rate 5e-5, 3e-5 or 2e-5 · 2–4 epochs
About 1 hour on one Cloud TPU

Four input–output patterns

(a) Sentence pair → class

`[CLS] A [SEP] B [SEP]`

Read **C** · MNLI, QQP

(b) One sentence → class

`[CLS] A [SEP]`

Read **C** · SST-2, CoLA

(c) Question → answer span

`[CLS] Q [SEP] passage [SEP]`

Read **every passage token** · SQuAD

(d) Sentence → tag per token

`[CLS] sentence [SEP]`

Read **every token** · NER

GLUE: 8 understanding tasks

Each task comes with its own human-labeled training data.

Sentence pair

MNLI: entails, contradicts or neutral?
QQP: are the two questions the same?
QNLI: does B answer question A?
STS-B: how similar, from 1 to 5?
MRPC: are they paraphrases?
RTE: does A entail B?

One sentence

SST-2: positive or negative review?
CoLA: is the sentence grammatical?

GLUE: fine-tune with one matrix

$p = \mathrm{softmax}(C\,W^{T}),\ W\colon K \times H$

```
[CLS] A man plays a guitar. [SEP] A person makes music. [SEP]
```

MNLI, K = 3 → **entailment**

Model	GLUE average
OpenAI GPT (previous best)	75.1
BERT-base	79.6
BERT-large	82.1

SQuAD v1.1: find the answer span

Q: Who released BERT?

Passage: BERT was released by **Google** in 2018. It was trained on books and Wikipedia.

```
P_start(i) = softmax over i of S · T_i
P_end(j) = softmax over j of E · T_j
answer = the span (i, j ≥ i) with the top S·T_i + E·T_j
```

New weights: only **two vectors**, S and E. Result: **90.9 F1** (dev, single model).

SQuAD 2.0: maybe no answer

Q: What is BERT's favorite color?

Passage: BERT was released by Google in 2018. …

Answer: none → point to [CLS]

```
s_null = S·C + E·C
s_best = top S·T_i + E·T_j (j ≥ i)
answer only if s_best > s_null + τ
```

τ is tuned on dev data. Result: **83.1 F1**, +5.1 over the previous best.

SWAG: pick the most likely ending

On stage, a woman takes a seat at the piano. She

a) sits on a bench as her sister plays with the doll.
b) smiles with someone as the music plays.
c) is in the crowd, watching the dancers.
d) nervously sets her fingers on the keys. (correct)

`4 inputs: [CLS] context [SEP] ending_k [SEP] → s_k = v · C_k`

BERT-large **86.3%** · OpenAI GPT 78.0% · humans 88.0%

NER: one label per token

Ada	Love	##lace	was	born	in	London
B-PER	I-PER	–	O	O	O	B-LOC

```
label_i = softmax(W T_i)
```

Without fine-tuning: use BERT as a **feature extractor**. Joining the top 4 layers gives **96.1 F1**, only 0.3 below fine-tuning.

What matters: the ablation

BERT-base, dev	MNLI	QNLI	MRPC	SQuAD
Full BERT	84.4	88.4	86.7	88.5
No NSP	83.9	84.9	86.5	87.9
Left-to-right, no NSP	82.1	84.3	77.5	77.8

The biggest loss comes from reading **left to right only**: −10.7 F1 on SQuAD.

BERT vs OpenAI GPT (2018)

	BERT	OpenAI GPT
Attention	Both ways	Left only
Pre-training	Masked LM + NSP	Next word
Data	Books + Wikipedia	Books only
[CLS], [SEP]	Learned in pre-training	Added at fine-tuning
Batch	128k words	32k words
GLUE score	80.5	72.8

Same data, same size: who wins?

Ettin (2025): encoders and decoders, trained on the same 2T tokens.

Encoders win

Classification and retrieval

A **150M** encoder beats a **400M** decoder on MNLI.

Decoders win

Generation

The lead **grows** with model size.

Converting one type into the other **does not catch up**.

BERT today: still everywhere

BERT, monthly downloads

68M

2nd on Hugging Face

All encoders, monthly

>1B

vs 397M for decoders

Used for: search and RAG retrieval · classification and moderation · entity extraction · safety classifiers beside LLMs · decision models (Von)

From BERT to today’s encoders

Model	Year	Main change
BERT	2018	Masked LM + NSP, 512 tokens
RoBERTa	2019	No NSP, more data, larger batches
ModernBERT	2024	RoPE, 8,192 tokens, 2T tokens with code
mmBERT	2025	1,800+ languages, 3T tokens
Ettin	2025	Open data; beats ModernBERT

Summary

1. BERT is an encoder that reads **both ways**.
2. Pre-training: **masked LM + NSP** on 3.3B words, with no labels.
3. Fine-tuning: **one small layer** on C, on every token, or as S and E vectors.
4. Bidirectionality gives the biggest gain over GPT-style training.
5. Today: encoders still handle most **classify, retrieve, extract** work.

Read more

Papers

BERT (Devlin et al., 2018)

SWAG (Zellers et al., 2018)

ModernBERT (Warner et al., 2024)

Ettin (Weller et al., 2025)

Blog posts

The Illustrated BERT (Alammar)

Understanding Encoder and Decoder LLMs (Raschka)

Introducing ModernBERT (Hugging Face)

Ettin Suite (Hugging Face)